

# A Beginner's Guide to the Duranta OpenAirInterface5G Codebase

*Repo: [duranta-project/openairinterface5g](https://github.com/duranta-project/openairinterface5g)*

This is Duranta OAI — a fork/continuation of the well-known OpenAirInterface (OAI) project. It's a full open-source software implementation of a cellular network: it can act as a 4G eNB, a 5G gNB (base station), a 4G/5G UE (phone/modem), and the various core-network protocol stacks around them. It's mostly written in C, uses CMake to build, and is meant to run on Linux, either with real radios (USRP, etc.) or with a built-in RF simulator (no hardware needed).

This guide is meant to orient a beginner: what each top-level folder is for, then a walkthrough of the specific files you listed.

## 1. The Big Picture — How a Cellular Stack Maps to Folders

Before diving into folders, it helps to know the mental model OAI uses. A cellular network is normally described in layers (like networking OSI layers, but 3GPP-specific):

- **Layer 1 (PHY)** — turns bits into radio waveforms and back (modulation, FFT, channel coding, etc.)
- **Layer 2 (MAC/RLC/PDCP/RRC/SDAP)** — scheduling, retransmission, ciphering, radio resource control
- **Layer 3 (NAS/NGAP/S1AP/GTP)** — talks to the "core network" (authentication, mobility, session setup)

OAI's folder layout mirrors this layered design almost exactly, which is why the top-level structure looks the way it does.

## 2. Top-Level Folder Structure

Here's what each top-level folder/file is for (this comes straight from the project's own doc/README.md and repo layout):

| Path           | What it is                                                                                                                |
|----------------|---------------------------------------------------------------------------------------------------------------------------|
| CMakeLists.txt | The master build recipe — defines every library and executable OAI can produce.                                           |
| cmake_targets/ | Build helper scripts (most importantly build_oai) and where compiled binaries end up after a build.                       |
| common/        | Shared utility code used everywhere: logging, config-file parsing, the "T tracer" debugging framework, thread pools, etc. |

| Path                 | What it is                                                                                                                                                                                                 |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| doc/                 | All documentation — tutorials, design notes, build instructions.                                                                                                                                           |
| docker/              | Dockerfiles for building OAI images (Ubuntu/RHEL based).                                                                                                                                                   |
| executables/         | The main() files — the actual entry points compiled into nr-softmodem, nr-uesoftmodem, lte-softmodem, etc. Best place to start reading code, since everything else is a library these files pull together. |
| nfapi/               | (n)FAPI — the standard interface between L1 (PHY) and L2 (MAC), useful when splitting them into separate processes.                                                                                        |
| openair1/            | Layer 1 (PHY) code — both LTE and NR physical layer, channel coding, modulation, radio unit (RU) procedures.                                                                                               |
| openair2/            | Layer 2 code — MAC, RLC, PDCP, RRC, SDAP, plus the various "AP" interfaces (F1AP, X2AP, E1AP, M2AP) used to split a gNB into CU/DU components.                                                             |
| openair3/            | Layer 3 / core-network-facing code — NAS, NGAP (5G), S1AP (4G), GTP tunneling, security functions.                                                                                                         |
| radio/               | Radio hardware drivers — USRP, BladeRF, the RF simulator, 7.2 fronthaul, ZMQ, etc. Whichever is picked becomes a shared library loaded at runtime.                                                         |
| targets/             | Sample configuration files for different deployment scenarios (this is where the two .conf files live). Historically also held build scripts — mostly superseded by cmake_targets now.                     |
| tests/               | Component-level test harnesses (e.g., a CU-UP tester, a NAS simulator).                                                                                                                                    |
| tools/               | Developer tooling — code formatting (clang-format), static analysis (cppcheck), include-checking (iwyu).                                                                                                   |
| ci-scripts/          | Scripts and configs used by the project's own Continuous Integration pipeline.                                                                                                                             |
| charts/ / openshift/ | Kubernetes/OpenShift deployment charts.                                                                                                                                                                    |

**Rule of thumb for navigating:** if you want to know what a message means, look in openair2/openair3 (the protocol layers). If you want to know how a radio signal is generated/processed, look in openair1. If you want to know how everything is wired together and started, look in executables.

### 3. CMakeLists.txt (top-level, ~2200 lines)

This is the root build script for the whole project, invoked automatically whenever you build (you rarely edit it by hand as a beginner). What it does, roughly top-to-bottom:

1. Project setup — sets the minimum CMake version, the C/C++ standards, and the OAI version number.
2. Compiler/CPU tuning — detects your CPU (x86\_64 vs ARM, and available SIMD extensions like AVX2/AVX512/GFNI) and sets matching compiler flags for fast signal processing. This matters a lot for OAI because the PHY layer is CPU-intensive.
3. Optional features as CMake options — things like `SANITIZE_ADDRESS`, `SANITIZE_THREAD`, `T_TRACER`, `E2_AGENT`, `TRACY_ENABLE`, `ENABLE_IMSCOPE` are all declared here as ON/OFF switches you can pass to the build.
4. Dependency handling — checks for/downloads the `asn1c` compiler (used to auto-generate C code from 3GPP ASN.1 message definitions — this is how RRC/NGAP/S1AP message structs get created), and pulls in a few external libraries via CPM (a CMake package manager).
5. Library definitions — this is the bulk of the file. Each protocol/component becomes a CMake `add_library(...)` target: `lte_rrc`, `nr_rrc`, `s1ap`, `ngap`, `m2ap`, `x2ap`, `flap`, `SECURITY`, `UTIL`, and the various PHY source-file groups (`PHY_SRC`, `PHY_NR_SRC`, `SCHED_NR_LIB`, etc.). Each `add_library` block lists the `.c` files that make it up and `target_link_libraries` that connect it to the libraries it depends on.
6. Further down (not shown in the excerpt above but present in the full file) it defines the actual executables — `nr-softmodem`, `nr-uesoftmodem`, `lte-softmodem`, the PHY simulators, etc. — by linking these libraries together.

**Beginner takeaway:** think of this file as a giant dependency graph. Almost every subsystem in `openair1/2/3` gets its own small library here, and the executables at the bottom are just "link these particular libraries together."

To open it run this:

```
~ cd ~/duranta/openairinterface5g
```

```
~ gvim CMakeLists.txt
```

If `gvim` is not installed run: `~ sudo apt install gvim`

## 4. cmake\_targets/build\_oai

This is a shell script wrapper around CMake — the tool most people actually run to build OAI (rather than calling cmake/make/ninja directly). It exists to make building less error-prone for newcomers.

What it handles:

- **-I** — installs all OS-level dependencies (compilers, libconfig, ASN1C, etc.) needed to build OAI. You typically run this once on a fresh machine.
- **Component selection flags** — `--gNB`, `--nrUE`, `--eNB`, `--UE`, `--RU` tell it which executables to actually build.
- **-w <HW>** — selects which radio driver to build/link against, e.g. USRP, SIMU (the RF simulator), BLADERF.
- **-g <mode>** — build type: Debug, Release, RelWithDebInfo, MinSizeRel.
- **--ninja** — use the faster Ninja build system instead of make.
- **--build-lib <name>** — build an optional shared library at runtime-loadable, e.g. telnetrv (a telnet control interface) or nrscope (a live signal visualizer).
- **--phy\_simulators** — builds the standalone PHY unit-test simulators (`nr_dlsim`, `nr_ulsim`, `ldptest`, etc.) instead of the full softmodem.
- **-c / --clean** — cleans previous build artifacts.

Under the hood, it essentially: creates a build directory (e.g. `cmake_targets/ran_build/build`), runs `cmake` with the right `-D` options translated from its own flags, then invokes `make/ninja`. After a successful build, your compiled binaries (`nr-softmodem`, `nr-uesoftmodem`, etc.) show up in `cmake_targets/ran_build/build/`.

If you wish to see where these tags are coming from, run this:

```
~ cd ~/duranta/openairinterface5g/cmake_targets
```

```
~ ./build_oai -help
```

The help flag list all the flags and option that we can pass as argument while running any exe file.

```

test@LAPTOP-931H1VKT:~/duranta/openairinterface5g/cmake_targets$ ./build_oai --help

This script compiles OpenAirInterface Software, and can install dependencies
for a number of distributions (Ubuntu 22-24, Fedora, RHEL9).
Options:
--arch-native
    Passes -march=native to the compiler.
--build-tool-opt
    Pass options to build tool (ninja/make), e.g. -k ("keep going")
-c | --clean
    Erase all files to make a rebuild from start
-C | --clean-all
    Erase all files made by previous compilations, installations
--clean-kernel
    Erase previously installed features in kernel: iptables, drivers, ...
--cmake-opt
    Pass the supplied option verbatim to cmake.
-d | --build-dir
    Sets build directory (will be <oai-root>/cmake_targets/<build-dir>/build)
--build-e2
    Enable the the E2 Agent
-I | --install-external-packages
    Installs required packages such as LibXML, asn1.1 compiler, ...
    This option will require root password
--install-optional-packages
    Install useful but not mandatory packages such as valgrind
-g | --run-with-gdb <Release | RelWithDebInfo | MinSizeRel | Debug>
    specify the build mode used by cmake. defaults to Debug mode if -g is used alone, with no mode parameter
    if -g is not specifies, Release mode is used.

```

A very common beginner command sequence looks like:

```

cd cmake_targets
./build_oai -I # install dependencies (once)
./build_oai -w SIMU --gNB --nrUE # build gNB + UE with RF simulator support

```

## 5. /doc

This folder is the documentation hub. doc/README.md is the index page — it links out to everything else, grouped into:

- **General** — FEATURE\_SET.md (what's implemented), GET\_SOURCES.md, BUILD.md, environment-variables.md.
- **Tutorials** — step-by-step guides such as "OAI 5GC", "OAI gNB with COTS UE", "OAI NR-UE", and "Multiple OAI NR-UE with RFsimulator" — these are the best starting points for actually running something end-to-end.
- **RUNMODEM.md** — the master reference for command-line options when launching the gNB/UE (simulators vs. hardware, MIMO, custom frequencies, etc.).
- **Design docs** — SW\_archi.md, F1/E1 interface design notes, nr-ue-design.md (explains the UE's L1 threading model), MAC/RRC usage notes.
- **Developer tools** — code-style-contrib.md, clang-format.md, sanitizer usage, cross-compilation for ARM.
- **Testing** — UnitTests.md, CI/test-bench documentation.

**Beginner takeaway:** if you're trying to actually run something (rather than just read code), `doc/` — specifically the tutorials and `RUNMODEM.md` — is where to go before digging into source.

## 6. executables/nr-softmodem.c — the gNB (5G base station) entry point

This file contains the `main()` function for the `nr-softmodem` binary — i.e., this is what you execute to run a 5G gNB. Conceptually, it does the following (this description reflects the well-established structure of this file across OAI releases; treat exact function/variable names as approximate since the file evolves):

- Command-line and config parsing — it calls into OAI's config module (`common/config`) to parse both command-line flags (like `--rfsim`, `--sa`, `-O <file>`) and the `.conf/.yaml` configuration file passed with `-O`. This populates global structures describing the gNB(s) to run (`RC.nb_nr_L1_inst`, `RC.nrrrc`, etc.).
- Logging & tracing setup — initializes the logging subsystem and, if enabled, the T tracer (a framework for recording detailed internal events for later analysis/visualization).
- Signal handling — installs handlers for signals like `SIGINT/SIGSEGV` so the process can shut down (relatively) gracefully and print useful debug info on a crash.
- Radio device / RU initialization — loads whichever radio driver was selected (real hardware such as USRP, or the RF simulator) as a shared library and initializes it.
- Stack initialization — brings up the layers in order: PHY (`openair1`), MAC/RLC/PDCP/RRC (`openair2`), and, if running in Standalone (SA) mode, the NGAP/GTP connection to the 5G core network (`openair3`). If `--phy-test` or `--do-ra` modes are requested, it configures those simplified test modes instead of the full protocol stack.
- Thread creation — spins up the real-time threads that do the actual per-subframe/slot work (RX, TX, scheduling), and then the main thread typically just waits (e.g. on the RU threads) until termination, occasionally handling the optional GUI/scope display.

**Beginner takeaway:** this file is the "conductor" — it doesn't do PHY math or protocol logic itself; it just parses configuration, then initializes and wires together the libraries built via `CMakeLists.txt` (the same ones from `openair1/2/3`), then starts the real-time loop.

Run this:

```
~ cd ~/duranta/openairinterface5g/executables
```

```
~ gvim nr-softmodem.c
```

Now to navigate inside it:

*Press /*

Type main and hit enter.

This is a general process of find and goto inside any file.

```
void init_eNB_afterRU(void);
configmodule_interface_t *uniqCfg = NULL;
int main( int argc, char **argv ) {
    int ru_id, CC_id = 0;
    start_background_system();

    ///static configuration for NR at the moment
    if ((uniqCfg = load_configmodule(argc, argv, CONFIG_ENABLECMDLINEONLY)) == NULL || CONFIG_ISFLAGSET(CONFIG_ABORT)) {
        exit_fun("[SOFTMODEM] Error, configuration module init failed\n");
    }

    set_softmodem_sighandler();
#ifdef DEBUG_CONSOLE
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
#endif
    mode = normal_ttrx;
    logInit();
    lock_memory_to_ram();
    get_options(uniqCfg);

    if (!has_cap_sys_nice())
        LOG_W(UTIL,
            "no SYS_NICE capability: cannot set thread priority and affinity, consider running with sudo for optimum performance\n");

    softmodem_verify_mode(get_softmodem_params());

#ifdef T_TRACER
    T_Config_Init();
#endif
}
nr-softmodem.c
/main
```

Now reading the files:

Press :

Type set tags=tags

Hit Enter.

Press /

Type DevAssert. Hit Enter.

This will take you to where DevAssert is written for the first time. Now press shift + 8 to find its other instances and if it's a user defined function, you can go to and read the function definition by this process.

## 7. executables/nr-uesoftmodem.c — the 5G UE (phone/modem) entry point

This is the `main()` for `nr-uesoftmodem`, the counterpart that emulates a 5G handset. Structurally it mirrors `nr-softmodem.c` closely, with UE-specific differences:

7. Config/CLI parsing — same config module, but reads UE-specific parameters: things like `-r` (number of PRBs), `--numerology`, `-C` (center frequency), `--band`, and UICC parameters (`--uicc0.imsi`, or a `-O ue.conf` file) that emulate the SIM card.
8. Cell search & synchronization — unlike the gNB (which just starts transmitting), the UE first has to find and synchronize to a cell: detecting the PSS/SSS synchronization signals and decoding the MIB/SIB broadcast information (this logic mostly lives in `openair1/PHY/NR_UE_TRANSPORT` and `nr_initial_sync.c`, called from here).
9. NAS/registration — once synchronized, it proceeds through Random Access, RRC connection setup, and (in SA mode) NAS registration/authentication with the 5G core, using the UICC credentials it was given.
10. Threading model — creates the UE's real-time processing threads (documented in `doc/nr-ue-design.md`), which run the receive chain, decode downlink control/data, and generate uplink transmissions in response.
11. RF device — same as the gNB: connects to either a real radio or the RF simulator (as a client this time, connecting to the gNB acting as server, or vice versa depending on config).

**Beginner takeaway:** `nr-uesoftmodem.c` looks almost like a mirror image of `nr-softmodem.c` — same config/logging/RF-init skeleton — but the actual protocol behavior is "client-like" (search → sync → random access → register) instead of "server-like" (broadcast → listen → schedule).

## 8. targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf

This is a UE configuration file — it plays the role of a SIM card profile. It's parsed by the config module and used by `nr-uesoftmodem` (via `-O ue.conf`) instead of passing everything on the command line. A typical `ue.conf` centers on a `uicc0` block containing:

- **imsi** — the UE's International Mobile Subscriber Identity (must match a subscriber record in the core network's database).
- **key** — the UE's shared secret authentication key (must match the core's records).
- **opc** — the operator variant algorithm configuration field, part of the Milenage authentication algorithm (again, must match the core).
- **dnn** (Data Network Name) — which "APN-equivalent" network the UE requests a data session on (e.g. "oai").
- **nssai\_sst** / **nssai\_sd** — Network Slice Selection Assistance Information: which network slice the UE should attach to.



**Beginner takeaway:** this file has nothing to do with radio parameters — it's purely identity/authentication data. Its values must exactly match whatever subscriber database your 5G core network (e.g. OAI CN5G) is using, or authentication will fail.

```
# SPDX-License-Identifier: LicenseRef-CSSL-1.0

uicc0 = {
  imsi = "999700000000001";
  key = "fec86ba6eb707ed08905757b1bb44b8f";
  opc = "C42449363BBAD02B66D16BC975D77CC1";
  pdu_sessions = ({ dnn = "oai"; nssai_sst = 1; });
}

position0 = {
  x = 0.0;
  y = 0.0;
  z = 6377900.0;
}

@include "channelmod_rfsimu_LE0_satellite.conf"
```

## 9. targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb.sa.band78.fr1.106PRB.pci0.rfsim.conf

This is a gNB configuration file, and its name tells you exactly what scenario it's for, piece by piece:

- **sa** — Standalone mode (gNB connects directly to a 5G core, no 4G anchor).
- **band78** — 3GPP band n78 (mid-band TDD, ~3.5 GHz), one of the most common 5G test bands.
- **fr1** — Frequency Range 1 (sub-6 GHz, as opposed to FR2 mmWave).
- **106PRB** — 106 Physical Resource Blocks of channel bandwidth (roughly 40 MHz at this numerology).
- **pci0** — Physical Cell ID 0 (distinguishes this cell from neighboring ones, relevant for multi-gNB/handover setups).
- **rfsim** — configured to run against the built-in RF simulator rather than real hardware.

Inside, this kind of file is organized into libconfig-style sections — a beginner reading it will typically find:

```

# SPDX-License-Identifier: LicenseRef-CSSL-1.0

Active gNBs = ( "gNB-OAI");
# Asn1_verbosity, choice in: none, info, annoying
Asn1_verbosity = "none";

gNBs =
(
{
    ////////// Identification parameters:
    gNB_ID      = 0xe00;
    gNB_name    = "gNB-OAI";

    // Tracking area code, 0x0000 and 0xffff are reserved values
    tracking_area_code = 1;
    plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2; snssaiList = ({ sst = 1; }) });
    @include "neighbour-config-rfsim.conf"

    nr_cellid = 12345678L;

    ////////// Physical parameters:

    do_CSIRS                                = 1;
    do_SRS                                  = "periodic";

    #uess_agg_levels = [0,1,2,2,1]
    servingCellConfigCommon = (
    {
#spCellConfigCommon

        physCellId                        = 0;

# downlinkConfigCommon
#frequencyInfoDL
    # this is 3600 MHz + 43 PRBs@30kHz SCS (same as initial BWP)
    absoluteFrequencySSB                  = 641280;
    dl_frequencyBand                       = 78;
    # this is 3600 MHz
    dl_absoluteFrequencyPointA             = 640008;
    #scs-SpecificCarrierList
    dl_offstToCarrier                      = 0;
# subcarrierSpacing
# 0=kHz15, 1=kHz30, 2=kHz60, 3=kHz120
    dl_subcarrierSpacing                  = 1;
    dl_carrierBandwidth                    = 106;
    #initialDownlinkBWP
    #genericParameters
    # this is RBstart=27,L=48 (275*(L-1))+RBstart
    initialDLBWPlocationAndBandwidth       = 28875; # 6366 12925 12956 28875 12952
# subcarrierSpacing
# 0=kHz15, 1=kHz30, 2=kHz60, 3=kHz120
    initialDLBWPsubcarrierSpacing         = 1;
gnb.sa.band78.frl.106PRB.pci0.rfsim.conf

```

+6

- **gNBs = ( { ... } )** — the core cell/gNB definition: PLMN identity (mcc, mnc), tracking\_area\_code, the physical cell parameters (subcarrier spacing/numerology, band, PRBs, absoluteFrequencySSB, dl\_absoluteFrequencyPointA), and slice info (sd, sst).
- **AMF connection parameters** — the IP address and other details for the 5G core's Access and Mobility Management Function, so the gNB knows where to send NGAP setup messages.
- **NETWORK\_INTERFACES** — local IP addresses the gNB uses for its N2 (control, to AMF) and N3 (user data, to UPF) interfaces.
- **A rfsimulator section (optional)** — parameters like serveraddr for the RF simulator, so you don't need to repeat them on the command line.

You'd typically run it like:

```
sudo ./nr-softmodem -O ../../../../targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb.sa.band78.fr1.106PRB.pci0.rfsim.conf --rfsim
```

**Beginner takeaway:** the filename encodes the "recipe" (mode/band/bandwidth/cell-id/hardware), and the file's content encodes cell identity + core-network connection details. The mnc/mnc/tracking\_area\_code/slice values here must match whatever your core network expects, the same way ue.conf's IMSI/key/opc must match the core's subscriber database.

## 10. How These Pieces Fit Together (Putting It All Together)

A minimal "hello world" for this codebase, once built, looks like:

- Build with `cmake_targets/build_oai` → produces `nr-softmodem` and `nr-uesoftmodem` binaries under `cmake_targets/ran_build/build/`.
- Start a 5G core network (separate project, e.g. OAI's CN5G, not in this repo).
- Run the gNB: `sudo ./nr-softmodem -O <gnb.conf> --rfsim` → this is `executables/nr-softmodem.c`'s `main()` parsing that `.conf` file and bringing up PHY/MAC/RRC/NGAP.
- Run the UE: `sudo ./nr-uesoftmodem -O ue.conf --rfsim` → `executables/nr-uesoftmodem.c`'s `main()` reads the IMSI/key from `ue.conf`, connects to the (simulated) radio, searches for and syncs to the gNB's cell, then registers with the core using those credentials.

That's essentially the whole story: `CMakeLists.txt` + `build_oai` produce the two programs; the `.conf` files tell each program who it is and how to identify its peers; and `openair1/2/3` are the libraries that do the actual protocol/signal-processing work these two `main()` files coordinate.

### Suggested Reading Order for a Newcomer

- `doc/README.md` and one of the RFsimulator tutorials — get something running first.
- `targets/PROJECTS/GENERIC-NR-5GC/CONF/*.conf` — understand the two config files above, since almost everything is driven by them.
- `executables/nr-softmodem.c` and `nr-uesoftmodem.c` — see the overall control flow.
- `doc/SW_archi.md` and `doc/nr-ue-design.md` — understand the threading/architecture in more depth.
- Dive into `openair1` (PHY) or `openair2/openair3` (protocols) depending on which layer interests you.